

Semana 4 — Clase Práctica en Laboratorio

Refactorización con Funciones y Diseño Modular

Introducción

Los problemas de esta semana marcan un cambio cualitativo respecto a las prácticas anteriores: el foco no está en resolver problemas nuevos desde cero, sino en transformar soluciones existentes en programas modulares bien diseñados. Los dos primeros bloques proponen la refactorización de problemas ya resueltos en las Semanas 2 y 3, aplicando funciones para ganar reutilización, legibilidad y testabilidad. El tercer bloque introduce un problema nuevo que debe resolverse directamente con diseño modular desde el análisis.

Cada problema sigue requiriendo la aplicación rigurosa de la **metodología de Pólya** (Análisis → Diseño → Codificación → Evaluación) y el **modelo de programa modular** presentado en la clase teórica: Sección Declarativa + Definición de Funciones + Sección Algorítmica (Prólogo / Resolución / Epílogo).

Los seis problemas están organizados en tres bloques con conexiones deliberadas: los Problemas 1 y 2 refactorizan soluciones de la Semana 2 (representaciones numéricas); los Problemas 3 y 4 modularizan sugerencias de profundización de la Semana 3 (algoritmo de la campesina rusa y números perfectos con primos de Mersenne); el Problema 5 requiere diseño modular original para un problema de teoría de números (sucesión alícuota).

Bloque A: Refactorización de soluciones de la Semana 2

Problema 1: Conversión de decimal a número romano (refactorizado)

Enunciado

Retomar la solución del Problema 3 de la Semana 2 (conversión de un entero positivo entre 1 y 3999 a números romanos) y refactorizarla aplicando funciones. El programa debe mantener exactamente el mismo comportamiento observable (mismas entradas, mismas salidas), pero su estructura interna debe ser modular.

Requisitos de modularización

El diseño debe incluir, como mínimo, las siguientes funciones:

digito_a_romano(digito, uno, cinco, diez): recibe un dígito (0–9) y los tres símbolos romanos correspondientes a su posición (por ejemplo, para las decenas: uno='X', cinco='L', diez='C'). Retorna la cadena romana equivalente. Esta función encapsula la lógica de los nueve casos (0–9) que se repetía cuatro veces en la versión original.

decimal_a_romano(numero): recibe un entero entre 1 y 3999, extrae los dígitos posicionales (millares, centenas, decenas, unidades), invoca digito_a_romano para cada posición con los símbolos apropiados, y retorna la cadena romana completa.

validar_rango(numero, minimo, maximo): retorna True si el número está en el rango [minimo, maximo], False en caso contrario. Función reutilizable para validación de entradas.

Casos de análisis y prueba

Los mismos de la Semana 2. Verificar que la refactorización no altera los resultados:

Número	Romano esperado	Observación
1	I	Mínimo
4	IV	Sustractivo unidades
9	IX	Sustractivo unidades
42	XLII	Decena + unidades
99	XCIX	Sustractivo decenas y unidades
399	CCCXCIX	Centenas máximas sin D
944	CMXLIV	Sustractivo en tres posiciones
2025	MMXXV	Año actual
3999	MMMCMXCIX	Máximo

Orientaciones para la resolución

Análisis: Identificar el código repetido en la solución original: la lógica de conversión de un dígito individual a romano se repite cuatro veces (una por posición), cambiando solo los símbolos. Esta repetición es la señal de refactorización.

Diseño: La función `digito_a_romano` usa una cadena de `if/elif` con ramas para los dígitos 1–3 (repeticiones), 6–8 (repeticiones), 4 (anteposición por resta), 5 y 9 (anteposición por resta); el 0 retorna cadena vacía. Los símbolos uno, cinco y diez son parámetros que permiten reutilizar la misma función para las cuatro posiciones.

Evaluación: Probar cada función de forma independiente antes de integrarlas. Por ejemplo, verificar que `digito_a_romano(4, 'I', 'V', 'X')` retorna 'IV' y que `digito_a_romano(4, 'X', 'L', 'C')` retorna 'XL'.

Reflexión: la potencia de la parametrización En la versión monolítica de la Semana 2, la lógica de conversión se copiaba cuatro veces con símbolos distintos. La función `digito_a_romano` convierte esa variación en parámetros: la estructura lógica se escribe una sola vez y los símbolos se pasan como argumentos. Si la Semana 2 descubría un error en la conversión del dígito 9, había que corregirlo en cuatro lugares; ahora basta corregirlo en uno.

Problema 2: Cardinalidad de un entero en palabras (refactorizado)

Enunciado

Retomar la solución del Problema 4 de la Semana 2 (expresar un entero positivo hasta 999.999.999 como cardinal en palabras) y refactorizarla aplicando funciones. El comportamiento observable debe ser idéntico.

Requisitos de modularización

El diseño debe incluir, como mínimo, las siguientes funciones:

unidades_a_texto(n): recibe un entero de 0 a 9 y retorna su cardinal en palabras ('', 'uno', 'dos', ..., 'nueve').

decenas_a_texto(n): recibe un entero de 0 a 99 y retorna su cardinal en palabras, manejando los casos especiales (10–15, 16–19, 20–29, 30–99).

grupo_a_texto(n): recibe un entero de 0 a 999 y retorna su cardinal completo, combinando centenas y decenas_a_texto. Maneja las irregularidades de las centenas (cien/ciento, quinientos, setecientos, novecientos).

numero_a_texto(n): recibe un entero positivo, lo descompone en grupos de tres dígitos (millones, miles, unidades), invoca grupo_a_texto para cada grupo y ensambla el resultado con los conectores apropiados ('mil', 'millón'/'millones').

Casos de análisis y prueba

Los mismos de la Semana 2. Verificar especialmente:

Número	Cardinal esperado	Tipo de caso
1	uno	Mínimo
15	quince	Especial (11–15)
21	veintiuno	Veinti- compuesto
100	cien	Centena exacta
101	ciento uno	Centena + unidad
500	quinientos	Centena irregular
1000	mil	Mil exacto (no 'un mil')
2025	dos mil veinticinco	Año actual
1000000	un millón	Millón singular
999999999	novecientos noventa y nueve millones novecientos noventa y nueve mil novecientos noventa y nueve	Máximo

Orientaciones para la resolución

Análisis: La solución de la Semana 2 ya identificaba tres subproblemas: (1) convertir un grupo de 0–999 a palabras, (2) aplicarlo a cada grupo, (3) ensamblar. La refactorización convierte cada subproblema en una función. Además, dentro del subproblema (1), las decenas tienen lógica propia que justifica una función separada.

Diseño: Las funciones se ordenan de menor a mayor nivel de abstracción: primero `unidades_a_texto`, luego `decenas_a_texto` (que puede invocar a la anterior), luego `grupo_a_texto`, y finalmente `numero_a_texto`. El programa principal solo invoca `numero_a_texto`.

Conexión con la Semana 2: diseño implícito vs. explícito Las orientaciones de la Semana 2 ya sugerían «descomponer en subproblemas» y «marcar claramente cada subproblema con comentarios». Ahora esa descomposición mental se materializa en funciones con contratos explícitos: lo que eran comentarios ahora son docstrings, y lo que eran bloques de código separados por líneas en blanco ahora son funciones invocables y testables independientemente.

Bloque B: Modularización de problemas sugeridos en la Semana 3

Problema 3: Multiplicación por el algoritmo de la campesina rusa

Enunciado

Implementar el **algoritmo de la campesina rusa** (también conocido como multiplicación egipcia o multiplicación por duplicación y mitad) para calcular el producto de dos enteros (positivos, negativos o cero). A diferencia del Problema 1 de la Semana 3 (multiplicación por sumas sucesivas con complejidad $O(b)$), este algoritmo reduce la complejidad a **$O(\log b)$** operaciones.

Requisito específico: para las operaciones de duplicación (multiplicar por 2) y división entera por 2, utilizar los **operadores de corrimiento de bits**: `<< 1` (corrimiento a la izquierda, equivale a $\times 2$) y `>> 1` (corrimiento a la derecha, equivale a $\div 2$). Esta conexión refuerza lo estudiado en la clase teórica sobre la representación binaria y las operaciones de la ALU.

Fundamento matemático

El algoritmo se basa en la descomposición binaria del multiplicador. En cada paso, se divide un operando por 2 (descartando el resto) y se duplica el otro. Cuando el operando que se divide es impar, se acumula el otro operando al resultado. El proceso termina cuando el operando dividido llega a 0.

```
resultado = 0
mientras b > 0:
    si b es impar: resultado = resultado + a
    a = a << 1      # duplicar a
    b = b >> 1      # mitad entera de b
```

Para averiguar si b es impar sin usar el operador `%`, basta con saber si el último dígito de su representación binaria es 1 con la condición `b & 1 == 1` (los operadores lógicos binarios a nivel de bits en Python son `&` -and-, `|` -or- y `^` -xor-). Por ejemplo, $10 \& 1 \rightarrow 0 \rightarrow 0$ ($0b1010 \& 0b0001$) y $15 \& 1 \rightarrow 1 \rightarrow 1$ ($0b1111 \& 0b0001$).

Requisitos de modularización

campesina_rusa(a, b): recibe dos enteros no negativos y retorna su producto usando el algoritmo descrito. Utiliza `<< 1` y `>> 1` para duplicar y dividir por 2.

multiplicar(a, b): recibe dos enteros cualesquiera (con signo), determina el signo del resultado, convierte a valores absolutos, invoca `campesina_rusa` y aplica el signo. Reutiliza la lógica de manejo de signos del Problema 1 de la Semana 3.

Casos de análisis y prueba

a	b	Producto	Pasos (duplicación/mitad)	Observación
13	11	143	a: $13 \rightarrow 26 \rightarrow 52 \rightarrow 104$; b: $11 \rightarrow 5 \rightarrow 2 \rightarrow 1$	Caso normal
3	4	12	a: $3 \rightarrow 6 \rightarrow 12$; b: $4 \rightarrow 2 \rightarrow 1$	$4 = 100_2$, acumula solo cuando b impar
7	0	0	0 pasos (b=0 inmediato)	Caso límite
0	5	0	Acumula 0 en cada paso	Caso límite

-3	4	-12	Signo aparte; opera con $ 3 \times 4 $	Signo negativo
1	999	999	~ 10 pasos ($\log_2 999 \approx 10$)	Eficiencia $O(\log b)$
255	255	65025	8 pasos ($\log_2 255 = 8$)	Caso simétrico

Verificación manual para $a = 13$, $b = 11$

```

a = 13, b = 11, resultado = 0
Paso 1: b=11 (impar) → resultado = 0+13 = 13;    a = 13<<1 = 26;    b = 11>>1 = 5
Paso 2: b=5 (impar) → resultado = 13+26 = 39;    a = 26<<1 = 52;    b = 5>>1 = 2
Paso 3: b=2 (par) → resultado = 39;              a = 52<<1 = 104;   b = 2>>1 = 1
Paso 4: b=1 (impar) → resultado = 39+104 = 143; a = 104<<1 = 208; b = 1>>1 = 0
b = 0 → fin. Resultado: 143 ✓ ( $13 \times 11 = 143$ )

```

Orientaciones para la resolución

Análisis: El algoritmo es un bucle while $b > 0$ que en cada iteración divide b por 2 y duplica a . La terminación está garantizada porque b se reduce estrictamente en cada paso (propiedad del corrimiento a la derecha sobre enteros positivos).

Diseño: La función `campesina_rusa` trabaja solo con no negativos (precondición). La función `multiplicar` maneja los signos y delega el cálculo. La condición de paridad se evalúa con $b \% 2 \neq 0$ o equivalentemente con $b \& 1$ (operación AND a nivel de bits; ambas son correctas).

Conexión holística: corrimientos, operaciones lógicas entre bits y la ALU Los operadores `&`, `<<` y `>>` de Python corresponden directamente a las instrucciones de **conjunción (and)** y de **corrimiento (shift) de la ALU** estudiadas en Arquitecturas de Computadoras. La conjunción del contenido de un registro de propósito general con una máscara sirve para desvelar los bits en las posiciones donde hay unos en la máscara. Un corrimiento a la izquierda agrega un 0 a la derecha de la representación binaria (equivalente a $\times 2$); un corrimiento a la derecha elimina el bit menos significativo (equivalente a $\div 2$ con truncamiento). El algoritmo de la campesina rusa es, de hecho, la versión elemental del **multiplicador binario por hardware** de la CPU: examina los bits de b uno a uno (mediante corrimiento) y acumula a desplazado a la posición correspondiente.

Problema 4: Números perfectos y primos de Mersenne

Enunciado

Implementar un programa modular que busque los primeros k **números perfectos** (donde k es ingresado por el usuario), verificando la relación con los **primos de Mersenne**. Para cada número perfecto encontrado, mostrar el número, el primo de Mersenne asociado y el exponente p tal que $2^p - 1$ es primo.

Fundamento: Euler demostró que todo número perfecto par tiene la forma $2^{p-1} \times (2^p - 1)$, donde $2^p - 1$ es un primo de Mersenne. Por lo tanto, la búsqueda se reduce a: para cada $p = 2, 3, 5, 7, 11, \dots$ (primos), verificar si $2^p - 1$ es primo; si lo es, $2^{p-1} \times (2^p - 1)$ es un número perfecto.

Requisitos de modularización

es_primo(n): retorna True si n es primo. Reutilizable en múltiples contextos.

mersenne(p): retorna el número de Mersenne $2^p - 1$ para un exponente p dado.

perfecto_desde_mersenne(p): dado un exponente primo p para el cual $mersenne(p)$ es primo, retorna el número perfecto correspondiente: $2^{p-1} \times (2^p - 1)$.

suma_divisores_propios(n): retorna la suma de los divisores propios de n (todos los divisores excepto n mismo). Se usa para verificar que el número perfecto encontrado cumple la definición: $suma_divisores_propios(n) == n$.

Casos de análisis y prueba

p	$2^p - 1$	¿Primo?	Número perfecto	Verificación
2	3	Sí	6	$1+2+3 = 6$ ✓
3	7	Sí	28	$1+2+4+7+14 = 28$ ✓
4	15	No (3×5)	—	—
5	31	Sí	496	suma div. propios = 496 ✓
7	127	Sí	8128	suma div. propios = 8128 ✓
11	2047	No (23×89)	—	—
13	8191	Sí	33550336	5.º número perfecto

Orientaciones para la resolución

Análisis: El programa principal itera sobre números primos p , prueba si $mersenne(p)$ es primo, y si lo es, calcula y muestra el número perfecto. Se detiene al encontrar k perfectos.

Diseño: Para iterar sobre primos, el programa puede usar un contador que avanza igual que para la búsqueda de posibles divisores para determinar si un número es primo pero sin tope centinela definido, y verifica primalidad con `es_primo`. Para $k = 4$ el programa termina rápidamente; para $k = 5$ ya requiere verificar la primalidad de 8191 (factible con el algoritmo básico de divisores hasta $\sqrt{n}$).

Conexión con la Semana 3 Este problema combina la función `es_primo` (que ya se presentó en la clase teórica de Semana 4 como ejemplo de abstracción funcional) con `suma_divisores_propios` (que encapsula el cálculo central del Problema 5 de la Semana 3 sobre números perfectos, deficientes y abundantes). La novedad es la composición de funciones: una función usa el resultado de otra como insumo, formando una cadena de procesamiento.

Bloque C: Problema nuevo con diseño modular

Problema 5: Sucesión alícuota de un entero

Enunciado

Escribir un programa modular en Python que solicite al usuario un **entero positivo $n \geq 2$** y muestre en una sola línea su **sucesión alícuota** hasta que se alcance el valor 1 (condición de terminación), se detecte un ciclo o se haya calculado una cantidad máxima de términos.

Fundamento matemático

La **suma alícuota** de un número n , denotada $s(n)$, es la suma de todos los divisores propios de n (es decir, todos los divisores de n excepto n mismo). Por ejemplo: $s(12) = 1 + 2 + 3 + 4 + 6 = 16$.

La **sucesión alícuota** de n es la secuencia: $n, s(n), s(s(n)), s(s(s(n))), \dots$ donde cada término es la suma alícuota del anterior. La sucesión puede:

1. **Terminar:** alcanzar un primo p (cuya suma alícuota es 1), seguido de 1. Ejemplo: $10 \rightarrow 8 \rightarrow 7 \rightarrow 1$.
2. **Entrar en un ciclo de período 1:** detenerse en un número perfecto ($s(n) = n$). Ejemplo: $6 \rightarrow 6 \rightarrow 6 \rightarrow \dots$
3. **Entrar en un ciclo de período 2:** alternar entre un par de números amigos. Ejemplo: $220 \rightarrow 284 \rightarrow 220 \rightarrow \dots$

Para los enteros que se solicitarán en esta práctica ($n \leq 1000$), todas las sucesiones terminan en 1, entran en un ciclo detectable en un número razonable de pasos o tienden a tener valores generalmente ascendentes.

Requisitos de modularización

suma_alicuota(n): recibe un entero $n \geq 2$ y retorna la suma de sus divisores propios. Nota: es la misma función `suma_divisores_propios` del Problema 4; si se resolvieron ambos, puede reutilizarse directamente.

mostrar_sucesion_alicuota(n): recibe un entero $n \geq 2$, calcula la sucesión alícuota invocando `suma_alicuota` iterativamente, y la muestra en una sola línea separada por flechas ($\rightarrow$). La función se detiene cuando el término actual es 1 (terminación) o cuando el término ya apareció antes en la secuencia (ciclo detectado).

Casos de análisis y prueba

n	Sucesión alícuota	Tipo de caso
10	$10 \rightarrow 8 \rightarrow 7 \rightarrow 1$	Termina en primo $\rightarrow 1$
12	$12 \rightarrow 16 \rightarrow 15 \rightarrow 9 \rightarrow 4 \rightarrow 3 \rightarrow 1$	Ascenso y luego descenso
6	$6 \rightarrow 6$ (ciclo: número perfecto)	Ciclo período 1
28	$28 \rightarrow 28$ (ciclo: número perfecto)	Ciclo período 1
220	$220 \rightarrow 284 \rightarrow 220$ (ciclo: números amigos)	Ciclo período 2
2	$2 \rightarrow 1$	Mínimo (primo)

3	$3 \rightarrow 1$	Primo
24	$24 \rightarrow 36 \rightarrow 55 \rightarrow 17 \rightarrow 1$	Secuencia ascendente-descendente
60	$60 \rightarrow 108 \rightarrow 172 \rightarrow 136 \rightarrow 134 \rightarrow 70 \rightarrow 74 \rightarrow 40 \rightarrow 50 \rightarrow 43 \rightarrow 1$	Secuencia larga (11 términos)
95	$95 \rightarrow 25 \rightarrow 6 \rightarrow 6$ (ciclo: llega a perfecto)	Terminación en ciclo de perfecto

Verificación manual para $n = 12$

```

n = 12
s(12) = 1+2+3+4+6 = 16
s(16) = 1+2+4+8 = 15
s(15) = 1+3+5 = 9
s(9) = 1+3 = 4
s(4) = 1+2 = 3
s(3) = 1
Sucesión: 12 → 16 → 15 → 9 → 4 → 3 → 1 ✓

```

Orientaciones para la resolución

Análisis: El núcleo del problema es un bucle que calcula iterativamente $s(n)$ y acumula los términos. La condición de corte tiene tres componentes: (a) el término actual es 1, (b) el término ya apareció en la secuencia (ciclo) o (c) se calculó una cantidad máxima de términos. Para la detección de ciclos, sin listas (que se introducen en la Semana 9), se puede limitar a comparar con el valor inicial y con el término anterior (lo cual detecta ciclos de período 1 y 2, suficiente para los casos de esta práctica).

Diseño: La función `suma_alicuota` recorre los posibles divisores de 2 a $\sqrt{n}$ y acumula los que dividen a n junto con el 1 inicial. La función `mostrar_sucesion_alicuota` construye la salida usando `print(..., end='')` para imprimir en una sola línea, o bien acumula el resultado en una cadena y lo imprime al final.

Conexión con teoría de números y con el curso La sucesión alícuota conecta varios conceptos: los **números perfectos** (Problema 5, Semana 3) son puntos fijos de $s(n)$; los **números amigos** son ciclos de período 2; y los primos son los «puntos de escape» que llevan a 1. La conjetura de Catalan-Dickson (1888, aún abierta) postula que toda sucesión alícuota es acotada: o termina en 1, o entra en un ciclo. El menor número cuyo destino se desconoce es **276** (cuya secuencia ha sido calculada por millones de términos sin resolverse).

Notas finales

Estos cinco problemas cubren los contenidos prácticos de la Semana 4 del curso. Los Problemas 1–2 ejercitan la refactorización (transformar código monolítico en modular sin alterar el comportamiento), los Problemas 3–4 practican la modularización de algoritmos con conexión directa al hardware (corrimientos de bits, multiplicación binaria), y el Problema 5 requiere diseño modular completo desde cero.

Criterios de evaluación formativa

Además de los criterios de semanas anteriores (metodología de Pólya, modelo de programa, *literate programming*), esta semana se evaluarán especialmente:

Cohesión: cada función realiza una sola tarea bien definida.

Acoplamiento: las funciones se comunican exclusivamente mediante parámetros y valores de retorno.

Contratos: cada función tiene una docstring que documenta precondition, postcondition y parámetros.

Testabilidad: cada función puede verificarse independientemente con casos de prueba específicos.

Reutilización: se aprovechan funciones de un problema en otro (por ejemplo, `es_primo` y `suma_divisores_propios/suma_alicuota` entre los Problemas 4 y 5).

Para profundizar

(1) En el Problema 1, extender para la conversión inversa: de romano a decimal, usando funciones separadas para validación y conversión. (2) En el Problema 3, comparar empíricamente la cantidad de operaciones del algoritmo de sumas sucesivas (Semana 3) con la campesina rusa, para distintos valores de b , y verificar la relación $O(b)$ vs. $O(\log b)$. (3) En el Problema 4, verificar que los 5 primeros números perfectos (6, 28, 496, 8128, 33550336) corresponden a los exponentes de Mersenne $p = 2, 3, 5, 7, 13$, y que $p = 11$ falla porque $2047 = 23 \times 89$ no es primo. (4) En el Problema 5, investigar la sucesión alícuota de 276 y determinar cuántos términos pueden calcularse antes de que los números excedan la capacidad práctica de cálculo. (5) Combinar los Problemas 4 y 5: un programa modular que clasifique los primeros 300 enteros según el destino de su sucesión alícuota (termina en 1, ciclo perfecto, ciclo amigo, o supera la cantidad máxima de términos calculados).